



Campus Connect

(MINOR PROJECT – II)

**SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE
OF**

**MASTER OF COMPUTER APPLICATIONS
(MCA)**

BY

PIYUSH GOEL

(Enroll. No.: 09311604425)

MCA – IInd Semester



**BHARATI VIDYAPEETH'S INSTITUTE OF
COMPUTER APPLICATIONS & MANAGEMENT (BVICAM)
GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY, NEW DELHI**

2026

CANDIDATE’S DECLARATION

I hereby declare that the work which is being presented in this project work entitled “**Campus Connect**” in partial fulfilment of the requirements for the award of the degree of **Master in Computer Applications at Bharati Vidyapeeth’s Institute of Computer Applications and Management (BVICAM), New Delhi** is an authentic record of my own work carried out during the period January 2026 to April 2026 under the supervision and guidance of **Ms. Saumya Bansal (Assistant Professor, BVICAM)**.

I have not submitted the matter embodied in this project work anywhere for the award of any degree or diploma.

Piyush

Enroll. No. 09311604425

ACKNOWLEDGEMENT

It is my proud privilege to express my profound gratitude to the entire management of Bharati Vidyapeeth's Institute of Computer Applications and Management and teachers of the institute for providing me with the opportunity to avail the excellent facilities and infrastructure. The knowledge and values inculcated have proved to be of immense help at the very start of my career. Special thanks to Hon'ble Founder, Bharati Vidyapeeth, Pune for having provided us an excellent infrastructure at BVICAM.

I am grateful to Ms. Saumya Bansal (Project Guide, BVICAM, New Delhi) for her astute guidance, constant encouragement and sincere support for this project work.

I feel pride and privileged in expressing my deep sense of gratitude to all those who have helped me in presenting this assignment. I would be failing in my endeavour if I do not place my acknowledgement.

Piyush Goel

Enroll. No. 09311604425

ABSTRACT

In this minor project, we have developed a Java based application for managing Alumni records of our college. The application is developed using Object Oriented Programming using Java. MySQL database management system (DBMS) is used to manage the database for the application. The development of application is carried out by following the Iterative and Prototyping model ensuring continuous refinement and user-centric improvements. The project consists of primarily 5 modules of the system: Dashboard, Static chat-bot, Community Chat, Memory Wall. All the modules are tested and verified to be functioning correctly in all respects. After successful testing, the system is implemented. This application aims to streamline the alumni data management, enhance the communication between the current students and the pass out alumni through formal channels and strengthen the alumni and institution connections through an interactive GUI.

TABLE OF CONTENTS

	Page No.
CHAPTER 1: INTRODUCTION	
1.1 Background	7
1.2 Problem Statement	7
1.3 Objective	8
1.4 Scope	8
1.5 Technology Used	9
CHAPTER 2: PROJECT DESCRIPTION	
2.1 System Specification	10
2.2 Methodology	10
2.2.1 Iterative Model	10
2.2.2 Prototyping Model	11
2.2.3 Requirement Analysis	12
2.2.4 System Design	12
2.2.5 Integration	12
2.2.6 Testing	12
2.4 Code	14
2.5 Assumptions & Dependencies	?
2.6 User Characteristics	?
CHAPTER 3: FUNCTIONALITIES	
3.1 Use Case Diagram	42
3.2 Data Flow Diagram(DFD 0)	43
3.3 Data Flow Diagram(DFD 1)	44
3.4 Entity-Relationship Diagram	45
3.5 Database Table	46

3.6	Screenshots	47
-----	-------------	----

CHAPTER 4: CONCLUSION AND FUTURE SCOPE

4.1	Conclusion	54
4.2	Future Scope	55

CHAPTER 1: INTRODUCTION

Background of Study:

The Student-Alumni communication is really important for the students to shape up their future and gain insightful guidance about how the industry works and what they'll be doing in specific roles in an Organization.

But, often students have to reach out to their seniors through informal channels such as Whatsapp, Instagram or linked-in which leads to limited reach, authenticity and communication gaps and takes up a lot of time from a student's busy schedule.

To counter such problems, we decided to create a app which bridges such gaps between the Students and Alumni. We thought "What if there was an application on which students can communicate with their seniors, see where they are working at and on what positions and ask one on one queries regarding the corporate life?".

So, we took up the opportunity to develop a java based application to create a networking bridge between the students and pass-out students.

Problem statement:

- The Campus Connect application provides a direct communication channel between the alumni and students through official channel
- Provides verified and valid information about alumni
- Provides a Structured platform with Static chatbot to solve queries of users, provide information regarding events and such.
- Community Chat for all Students and Alumni to talk as a community discuss ideas and have a little fun away from there hectic life.

Proposed Solution:

- The Campus Connect application provides a direct communication channel between the alumni and students through official channel
- Provides verified and valid information about alumni
- Provides a Structured platform with BVICAM Virtual Assistant to solve queries of users, provide information regarding events and such.
- Community Messages for all Students and Alumni to talk as a community discuss ideas and have a little fun away from there hectic life.

Team Structure and Role

- **Hirik Anand:** Database & Backend Logic
- **Varun Rawat:** UI/UX Design (HTML,JS)
- **Rudransh Singh Rawat:** AI Chatbot Integration
- **Piyush Goel:** Backend , Module Integration
- **Saksham:** Module Integration & Debugging

Objective:

- To create a JAVA based interactive application that connects students with alumni
- To create a well maintained and structured Data Base using MySQL and connect it with our JAVA app using JDBC.
- An easy to use Dashboard with all modules linked to.
- A login page for different level of access to the Campus Connect application.
- Provide various communication channels such as community chat and private one-on-one chat.
- A quick and responsive Chat Bot to solve small queries of the users of Campus Connect.
- To Provide a memory wall for Alumni to share their experience and stories on for everyone to inspire from.

Scope of Campus Connect:

- The system is made to use by Students, Alumni and Admin.
- The application contains various modules namely Dashboard, Static chatbot, Community Chat, Memory Wall, Feedback, Alumni profile, login page.
- The system lets students to have one-on-one communication with seniors, chatting with others as a community, upload pictures and stories on the memory wall, Static chat-bot to solve small queries and such.
- The system at this stage of development doesn't let its users to communicate through voice chats or video chats.
- The Campus Connect right now is not accessible globally, it only works at local level.

Technologies Used:

- **JAVA:** The project Campus connect was built mainly using JAVA for frontend and functioning.
- **JavaScript:** JavaScript is a high level language which is used to develop interactive and dynamic content on websites.
- **IntelliJ IDE:** a free and open-source IDE which was used to develop the Campus Connect application.
- **HTML 5:** It is a markup language consisting of tags, used to create structured web pages.
- **Java SprinBoot**
- **Maven:** It is an automation and project management tool primarily used for java applications.

Assumptions and Dependencies:

Anyone with latest web browser and network support can access out site.

User Characteristics:

- **Technical Knowledge:** the user must have basic computer literacy such as using a mouse, keyboard and should be familiar with JAVA, NetBeans and MySQL.
- **Domain knowledge:** The users using this app should have basic domain knowledge and understand the purpose of the application Campus Connect.

CHAPTER 2: PROJECT DESCRIPTION

System Specifications:

- i. Hardware Requirements:
 - a) Operating system: Windows 7-11, Linux
 - b) Intel, AMD or Apple silicon with minimum 2 cores and 2 threads
 - c) System Memory: At least 4 GB RAM
 - d) Storage: At least 10 GB Free space.
- ii. Software Requirements:
 - a) JDK 25
 - b) Apache Netbeans
 - c) Apache Maven
 - d) Spring Boot

Methodology:

1. Agile Model:

The Agile model is an object oriented software engineering approach in which the development of the project occurs in various steps (Iterations) instead of developing the entire project once. Our team have used this approach so that we can develop the small parts or modules test them and then fit them with everything else in our project and then focus on the next part and constant customer feedback.

We followed these steps Designing→ Coding →testing and then implementing into our project.

Phases we went through while developing this project through iterative model

- Planning and Requirement analysis: Firstly, we thought of the problems that may occur when students try to reach out to their seniors and alumni.
- System Design: Then we designed the UI for the modules.
- Implementation: We implemented the modules using Java and MySQL for database.
- Testing: We tested the modules for errors and improvements.
- Evaluating: We then evaluated whether the module passed our expectations or not.

Why we used the Agile model:

- In our project we came up with multiple modules we wanted to add to the final application.
- It was easy to test and debug the errors in segments rather than finish the whole project and then counter the errors.
- It was easy to evaluate and test the modules using the Iterative model which made it easier to manage and find problems early on in the development cycle.

2. Requirement Analysis:

- Initially we gathered requirements for User Authentication, Alumni directory and revised and added more requirements as project was being developed.
- Once basic framework regarding the project was set we decided on including various modules like Memory Wall, Community Chat, Alumni Profile, and Static Chat-bot in our project.

3. System Design:

- Once we were happy with our prototypes we formed a detailed System Design.
- Data base design: we formed our data base design with the help of ER diagrams, UML diagrams like use case diagram, sequence diagram, dfd 0 and dfd 1 diagrams and such.

4. Integration:

- The individual modules were then integrated into a single functioning system named Campus Connect.
- Database connectivity was established.
- System level testing was done after the integration was completed.

5. Testing:

- The modules were tested during each iteration.
- Once all modules were integrated we tested the system.

Id	Test Case Description	Test Case Input	Test Results		Status	Corrective Measure
			Expected	Actual		
1	Alumni can successfully upload there memories/pictures..	Pictures.	Memory wall created and displayed	Pictures were uploaded and Memory wall was created	Pass	None
2	Whether the community chat works.	Different messages by Alumni and users.	Transfer of messages	Successfully transfer of messages	Pass	None
3	Functioning of Portal Login	Login Password	Successful Login	Successful Login	Pass	None

Repository Structure:

- Campus Connect Project Analysis

1. Overview

Campus Connect is a Spring Boot-based campus networking platform for BVICAM students, alumni, and administrators. The application combines JWT-secured REST APIs, Spring Security, JPA/Hibernate persistence, and Spring WebSockets/STOMP for real-time chat. The browser side is a lightweight HTML/CSS/Vanilla JavaScript frontend that loads tab-based components dynamically inside a single dashboard.

The codebase is organized around a clear product idea: one private campus community that supports authentication, profile management, user search, events, jobs, posts, feedback, admin operations, and both public and private messaging.

2. Repository Structure

- Backend
 - `src/main/java/com/bvicam/campusconnect/CampusConnectApplication.java` - Spring Boot entry point.
 - `config/` - Security, WebSocket, JWT filter, and data seeding configuration.
 - `controller/` - REST and STOMP endpoints for authentication, admin, users, events, jobs, posts, feedback, chat, and a small bot endpoint.
 - `service/` - Business logic for posts, events, email broadcasts, and user lookup.
 - `repository/` - Spring Data JPA repositories.
 - `entity/` - Database entities and enums.
 - `dto/` - Request/response objects used by controllers and the UI.
 - `util/` - JWT helper.

3. Frontend

- `src/main/resources/static/index.html` - Landing page.
- `src/main/resources/static/login.html` - Login page.
- `src/main/resources/static/dashboard.html` - Single-page dashboard shell.

- ``src/main/resources/static/components/`` - HTML fragments loaded into the dashboard.
- ``src/main/resources/static/js/`` - Core app bootstrap and feature modules.

4. Supporting Files

- ``pom.xml`` - Maven build and dependency definition.
- ``application.properties`` - Local database, mail, and JPA configuration.
- ``README.md`` - High-level project description.
- ``src/test/java/.../CampusConnectApplicationTests.java`` - Basic Spring context test.
- ``bulk1.xlsx``, ``bulk1 - Copy.xlsx`` - Sample bulk-upload data files.

5. Architecture Summary

The project follows a conventional layered Spring architecture:

- Controllers expose HTTP and WebSocket endpoints.
- Services contain business logic and transactional behavior.
- Repositories encapsulate persistence operations.
- Entities model the relational schema.
- DTOs keep request and response payloads separated from entities where needed.
- The frontend uses ``fetch``, JWT stored in ``localStorage``, and dynamic component loading inside ``dashboard.html``.

This is not a full SPA framework application. Instead, the dashboard acts like a shell that swaps HTML fragments and initializes module scripts on demand.

6. Backend Deep Dive

➤ Application Startup

``CampusConnectApplication`` enables Spring Boot auto-configuration and async execution. Startup is supplemented by ``DataSeeder``, which creates initial users and departments if they do not exist.

7. Security

- ``SecurityConfig`` sets up:

- Stateless session policy.
- JWT filter insertion before username/password authentication.
- Permitted static resources and authentication endpoints.
- CORS with permissive origin patterns.
- BCrypt password encoding.

`JwtAuthenticationFilter` reads the `Authorization: Bearer <token>` header, parses the token with `JwtUtil`, loads the user through `CustomUserDetailsService`, and places the authenticated user into the security context.

Important detail: role-based authorization is only partially enforced at the security layer. Most role checks happen inside controller or service methods rather than through explicit route-level security rules.

8. Authentication Flow

`AuthController` supports:

- `POST /api/auth/register` - Creates a user, sets the role, optionally attaches a department, and hashes the password.
- `POST /api/auth/login` - Authenticates credentials and returns a JWT plus user name and role.
- `POST /api/auth/change-password` - Updates the stored password and marks the password as changed.

The login response currently returns token, name, and role. The frontend stores these values in `localStorage` and uses the token for subsequent requests.

9. Admin Operations

`AdminController` is the most feature-dense backend controller. It supports:

- Listing all users.
- Creating users with a default password.
- Updating user profiles.
- Resetting passwords.
- Deleting users.
- Deleting posts.

- Broadcasting emails to all users.
- Bulk user upload from CSV or Excel.

The bulk upload logic uses Apache POI for `.xlsx` files and a simple CSV parser for `.csv` files. Imported records default to the password `'Bvicam@2025'` and `'passwordChanged=false'`.

10. User Directory and Profile

`'UserController'` provides:

- `'GET /api/users/me'` - Returns the current authenticated user.
- `'GET /api/users'` - Returns the searchable directory view.
- `'GET /api/users/search'` - Searches by role, batch, and keyword.
- `'PUT /api/users/profile'` - Updates the current user's profile fields.

The directory response is mapped into `'UserProfileDto'` to avoid exposing the full entity shape to the browser.

11. Events

`'EventController'` supports event lifecycle and RSVP logic:

- `'GET /api/events'` - Returns all events with calculated attendance state and participant count.
- `'POST /api/events'` - Creates an event.
- `'PUT /api/events/{id}'` - Updates an event.
- `'POST /api/events/{id}/rsvp'` - Toggles attendance for the authenticated user.
- `'GET /api/events/{id}/participants'` - Returns participant details.
- `'DELETE /api/events/{id}'` - Deletes an event.

`'EventService'` duplicates some of the same operations, but the controller currently owns the main request flow.

12. Jobs

`'JobController'` exposes:

- `'GET /api/jobs'` - Lists jobs newest-first.
- `'POST /api/jobs'` - Allows alumni and admins to post jobs.

- ``DELETE /api/jobs/{id}`` - Allows the poster or admin to delete.

The controller checks role names directly, which is simple but less robust than a centralized authorization policy.

13. Posts / Memory Wall

``PostController`` and ``PostService`` implement the wall feed:

- Posts are returned pinned-first, then newest-first.
- Users can create posts and attach an optional image URL.
- Likes are incremented with a dedicated endpoint.
- Admins can pin or unpin posts.
- Posts can be deleted by the author or an admin.

The feed is ordered by ``isPinned`` and ``createdAt``, so the pinning feature directly affects the presentation layer.

14. Feedback

``FeedbackController`` lets authenticated users submit feedback and lets admins read all feedback newest-first. The frontend exposes this through a modal rating widget.

15. Chat and Messaging

``ChatController`` handles both public and private messaging.

Public chat:

- ``@MessageMapping("/chat.sendMessage")`` broadcasts to ``/topic/public``.
- ``@MessageMapping("/chat.addUser")`` also broadcasts join notifications.

Private chat:

- ``@MessageMapping("/chat.private")`` stores a ``PrivateMessage`` and sends it to the sender and receiver queues.
- ``@MessageMapping("/chat.typing")`` relays typing state to the target user.
- ``GET /api/messages/history`` returns the conversation history between two users.
- ``GET /api/messages/partners`` returns recent conversation partners.

The implementation uses email addresses as the chat identity rather than a separate messaging handle.

Bot Endpoint

`BotController` is a rule-based helper endpoint. It inspects the question text and returns canned answers for:

- placement stats,
- batch lookups,
- contact info,
- syllabus references,
- upcoming events.

This is a lightweight FAQ bot, not an ML-powered assistant.

Domain Model

`User`

The central entity. It stores:

- identity and login fields: email, password hash, name, role,
- academic data: enrollment number, batch year, department,
- professional data: skills, headline, company, designation, experience, links,
- chat relationships and events.

It is carefully protected with `@JsonIgnore` on password and recursive associations.

`Department`

Simple lookup entity with unique `name` and `code`. Used to classify users and target content.

`Event`

Represents campus events, with:

- title, description, date/time, location, organizer,
- target department,
- many-to-many participants.

Equality is based on `id` only, which avoids recursion and collection issues.

`Job`

Represents an internship or job post with title, company, location, description, apply link, poster, timestamp, and optional target department.

`Post`

Represents wall updates. Fields include content, optional image URL, author, creation time, like count, and pinned state.

`Feedback`

Captures rating, comments, submitter, and timestamp.

`PrivateMessage`

Stores persistent private message history using sender and receiver email addresses plus timestamp.

`ChatMessage`

Looks like a richer chat entity with sender and recipient relationships. In practice, the active private chat persistence path uses `PrivateMessage`, while public/private websocket payloads use the DTO `ChatMessage`.

`Role`

Enum with `STUDENT`, `ALUMNI`, and `ADMIN`.

DTO Layer

The DTOs are used to keep browser-facing payloads simple and to reduce entity leakage.

- `RegisterRequest` - Registration input.
- `LoginRequest` - Login input.
- `AuthResponse` - Login success response.
- `UserProfileDto` - Editable profile data shown in the UI.
- `EventDTO` - Event list response with computed `attending` and `participantCount`.

- ``com.bvicam.campusconnect.dto.ChatMessage`` - WebSocket message payload for chat.
- ``TypingMessage`` - Typing-indicator payload.

Persistence Layer

Repositories are mostly standard Spring Data interfaces with a few custom queries:

- ``UserRepository.searchUsers(...)`` supports role, batch, and keyword search.
- ``EventRepository.findByDateTimeAfterOrderByDateTimeAsc(...)`` is used for upcoming events.
- ``PostRepository.findAllByOrderByIsPinnedDescCreatedAtDesc()`` supports feed ordering.
- ``JobRepository.findAllByOrderByPostedAtDesc()`` supports reverse chronological job listing.
- ``FeedbackRepository.findAllByOrderBySubmittedAtDesc()`` supports admin review.
- ``PrivateMessageRepository.findConversation(...)`` returns one-on-one chat history.

Frontend Deep Dive

Landing Page

``index.html`` is a marketing-style public homepage. It presents:

- the platform mission,
- feature cards,
- call-to-action buttons,
- college branding,
- contact and footer information.

Login Page

``login.html`` is a simple centered form. The JavaScript in ``js/app.js`` posts credentials to ``/api/auth/login``, stores token and user info in ``localStorage``, then redirects to the dashboard.

Dashboard Shell

``dashboard.html`` is the main authenticated experience. It contains:

- a left navigation rail,
- a central content panel that swaps components,

- a global chat column on desktop,
- modals for direct chat, job creation, profile viewing, admin editing, event editing, feedback, participants, and bot interaction.

This layout is intentionally modular. Most feature UI is loaded as fragments into the `main-content` region.

Dashboard Bootstrap

`js/dashboard-main.js` is the entry coordinator:

- checks local storage for the current user,
- redirects to login if missing,
- reveals admin-only navigation,
- connects WebSocket chat,
- loads the default tab,
- handles tab switching and component loading,
- exposes logout behavior.

API Helper

`js/utils/api.js` centralizes authenticated requests. It attaches the JWT to outgoing requests and redirects to login if a request returns `401`.

Feature Modules

- `directory.js` - Searchable directory and profile popups.
- `chat.js` - Global chat, private chat, typing indicators, recent chats.
- `events.js` - Event listing, RSVP, admin create/edit/delete, participants modal.
- `jobs.js` - Job listing, posting, deletion.
- `wall.js` - Feed rendering, create/delete/like/pin post logic.
- `profile.js` - Load and save the current user profile.
- `feedback.js` - Star-based feedback submission.
- `admin.js` - Admin user management, bulk upload, password reset, feedback view.

Main User Journeys

Authentication and Session

1. User logs in through `login.html`.
2. JWT, name, role, and email are stored in `localStorage`.
3. Dashboard bootstrap checks for a saved session.
4. All API calls include the token in `Authorization`.

Directory Search

1. User opens the directory tab.
2. JavaScript calls `/api/users/search` with role, batch, and keyword filters.
3. Results are rendered into a table.
4. A user can open a profile card or start a direct chat.

Private Messaging

1. Dashboard connects to `/ws` using SockJS and STOMP.
2. Messages are sent to `/app/chat.private`.
3. Backend stores the message and forwards it to sender and receiver queues.
4. Frontend loads conversation history from REST endpoints and live updates via WebSocket subscriptions.

Event RSVP

1. Events are fetched from `/api/events`.
2. The backend calculates attendance and participant count.
3. RSVP toggles call `POST /api/events/{id}/rsvp`.
4. Admins can view participants and edit or delete events.

Feed Posting

1. User writes a wall post.
2. Frontend posts to `/api/posts`.
3. Feed renders pinned posts first.
4. Admins can pin/unpin or delete posts.

Admin Bulk Onboarding

1. Admin opens the user management tab.
2. Uploads a CSV or Excel file.
3. Backend parses the file and creates users with defaults.

4. The UI refreshes the user table after upload.

Strengths

- Clear separation of controllers, services, repositories, entities, and frontend modules.
- Real-time chat support with both public and private channels.
- A practical admin workflow for onboarding users and managing content.
- DTO usage where the UI needs computed fields or a safer payload shape.
- Seed data that helps new installs show meaningful content immediately.
- The frontend is organized into feature modules instead of a single large script.

Risks and Caveats

- Some authorization checks are implemented manually inside controllers and services rather than centrally in security rules.
- The project mixes entity-backed JSON responses with DTO-backed responses, which can make frontend contracts less uniform.
- There are overlapping concepts in chat persistence: ``ChatMessage`` entity, ``PrivateMessage`` entity, and ``ChatMessage`` DTO.
- ``JwtUtil`` stores the secret in source code, which is not suitable for production.
- ``application.properties`` contains default local credentials and placeholder mail values.
- The frontend depends heavily on ``localStorage``, so session corruption or stale data can create confusing states.
- The dashboard uses global ``window`` functions for many actions, which works but is hard to scale.
- I could not complete a local Maven verification from this environment because ``mvn`` was not installed, and the wrapper command did not provide a usable build transcript here.

Build and Runtime Notes

- Target runtime is Java 17 with Spring Boot and MySQL-compatible storage.
- WebSocket endpoint is ``/ws``.
- REST APIs are under ``/api/...``.
- Static assets are served from ``src/main/resources/static``.
- The sample data and dashboard assume seeded users and departments exist.

Suggested Next Steps

- Move secrets such as JWT signing keys and mail credentials out of source and into environment variables.
- Consolidate chat persistence so there is one clear model for private message history.
- Add centralized role-based authorization for sensitive endpoints.
- Replace manual `window`-based client wiring with a more maintainable module pattern if the frontend grows further.
- Add integration tests for authentication, events, posts, and WebSocket chat.

Conclusion

Campus Connect is a feature-rich, monolithic campus community app with a strong demo-friendly scope. The backend covers authentication, directory search, events, jobs, posts, feedback, and real-time messaging, while the frontend presents them through a modular dashboard shell. The code is already fairly complete for a student/alumni portal, but production hardening would require better secret handling, tighter authorization, and more test coverage.

Code:

Admin Controller:

```
package com.bvicam.campusconnect.controller;

import com.bvicam.campusconnect.dto.RegisterRequest;
import com.bvicam.campusconnect.entity.User;
import com.bvicam.campusconnect.entity.Role;
import com.bvicam.campusconnect.repository.PostRepository;
import com.bvicam.campusconnect.repository.UserRepository;
import com.bvicam.campusconnect.service.EmailService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.*;
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.springframework.web.multipart.MultipartFile;

import java.io.BufferedReader;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final EmailService emailService;
```

```

private final PostRepository postRepository;

@Autowired
public AdminController(UserRepository userRepository,
                       PasswordEncoder passwordEncoder,
                       EmailService emailService,
                       PostRepository postRepository) {
    this.userRepository = userRepository;
    this.passwordEncoder = passwordEncoder;
    this.emailService = emailService;
    this.postRepository = postRepository;
}

// =====
// 1. USER MANAGEMENT (CRUD)
// =====

@GetMapping("/users")
public ResponseEntity<List<User>> getAllUsers() {
    return ResponseEntity.ok(userRepository.findAll());
}

@PostMapping("/users")
@Transactional
public ResponseEntity<?> createUser(@RequestBody RegisterRequest request) {
    try {
        if (userRepository.existsByEmail(request.getEmail())) {
            return ResponseEntity.badRequest().body("Error: Email already exists!");
        }

        User user = new User();
        user.setName(request.getName());
        user.setEmail(request.getEmail());
        // Safe Enum assignment from Version 2

```

```

        user.setRole(request.getRole() != null ? request.getRole() : Role.STUDENT);
        user.setEnrollmentNumber(request.getEnrollmentNumber());
        user.setBatchYear(request.getBatchYear());

        // Set Professional Profile Fields
        user.setHeadline(request.getHeadline());
        user.setCurrentCompany(request.getCurrentCompany());
        user.setDesignation(request.getDesignation());
        user.setSkills(request.getSkills());
        user.setGithubUrl(request.getGithubUrl());
        user.setLinkedInUrl(request.getLinkedInUrl());
        user.setPastExperience(request.getPastExperience());

        String defaultPass = "Bvicam@2025";
        user.setPasswordHash(passwordEncoder.encode(defaultPass));
        user.setPasswordChanged(false);

        userRepository.save(user);
        return ResponseEntity.ok("✔ User created successfully!");
    } catch (Exception e) {
        return ResponseEntity.status(500).body("Error creating user: " + e.getMessage());
    }
}

@DeleteMapping("/delete-user/{id}")
@Transactional
public ResponseEntity<?> deleteUser(@PathVariable Long id) {
    if (!userRepository.existsById(id)) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("User not found");
    }
    userRepository.deleteById(id);
    return ResponseEntity.ok("✔ User deleted successfully.");
}

```

```

    @PutMapping("/users/{id}")
    @Transactional
    public ResponseEntity<?> updateUser(@PathVariable Long id, @RequestBody User
updatedData) {
        User user = userRepository.findById(id).orElseThrow(() -> new
RuntimeException("User not found"));

        user.setName(updatedData.getName());
        user.setEmail(updatedData.getEmail());
        user.setRole(updatedData.getRole());
        user.setBatchYear(updatedData.getBatchYear());
        user.setEnrollmentNumber(updatedData.getEnrollmentNumber());
        user.setCurrentCompany(updatedData.getCurrentCompany());
        user.setDesignation(updatedData.getDesignation());
        user.setHeadline(updatedData.getHeadline());
        user.setSkills(updatedData.getSkills());
        user.setGithubUrl(updatedData.getGithubUrl());
        user.setLinkedInUrl(updatedData.getLinkedInUrl());
        user.setPastExperience(updatedData.getPastExperience());

        userRepository.save(user);
        return ResponseEntity.ok("✔ User updated successfully.");
    }

    @PutMapping("/users/{id}/reset-password")
    @Transactional
    public ResponseEntity<?> resetPassword(@PathVariable Long id) {
        User user = userRepository.findById(id).orElseThrow(() -> new
RuntimeException("User not found"));
        user.setPasswordHash(passwordEncoder.encode("Bvicam@2025"));
        user.setPasswordChanged(false);
        userRepository.save(user);
    }

```

```

        return ResponseEntity.ok("✔ Password reset to 'Bvicam@2025'");
    }

// =====
// 2. MODERATION & BROADCAST (Restored from Version 1)
// =====

@PostMapping("/broadcast")
public ResponseEntity<?> sendBroadcast(@RequestBody Map<String, String>
payload) {
    String subject = payload.get("subject");
    String body = payload.get("body");
    if (subject == null || body == null) {
        return ResponseEntity.badRequest().body("Subject and body are required");
    }
    emailService.sendBroadcastEmail(subject, body);
    return ResponseEntity.ok("Broadcast started in background!");
}

@DeleteMapping("/delete-post/{id}")
@Transactional
public ResponseEntity<?> deletePost(@PathVariable Long id) {
    if (!postRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
    postRepository.deleteById(id);
    return ResponseEntity.ok("Post deleted by Admin.");
}

// =====
// 3. BULK UPLOAD (Optimized Logic)
// =====

```

```

@PostMapping("/upload-users-excel")
@Transactional
public ResponseEntity<?> uploadUsersUniversal(@RequestParam("file")
MultipartFile file) {
    if (file.isEmpty()) return ResponseEntity.badRequest().body("Please select a file.");
    String fileName = file.getOriginalFilename();
    if (fileName == null) return ResponseEntity.badRequest().body("Invalid file.");

    try {
        if (fileName.toLowerCase().endsWith(".csv")) {
            return processCSV(file);
        } else if (fileName.toLowerCase().endsWith(".xlsx")) {
            return processExcel(file);
        } else {
            return ResponseEntity.badRequest().body("✗ Unsupported format. Please
upload .csv or .xlsx");
        }
    } catch (Exception e) {
        return ResponseEntity.status(500).body("Error during upload: " +
e.getMessage());
    }
}

private ResponseEntity<?> processExcel(MultipartFile file) throws Exception {
    try (InputStream is = file.getInputStream(); Workbook workbook = new
XSSFWorkbook(is)) {
        Sheet sheet = workbook.getSheetAt(0);
        int count = 0;
        for (int i = 1; i <= sheet.getLastRowNum(); i++) {
            Row row = sheet.getRow(i);
            if (row == null) continue;

            String name = getCellValue(row.getCell(0));

```

```

        String email = getCellValue(row.getCell(1));
        if (name.isEmpty() || email.isEmpty()) continue;

        saveUserFromData(name, email,
            getCellValue(row.getCell(2)), getCellValue(row.getCell(3)),
            getCellValue(row.getCell(4)), getCellValue(row.getCell(5)),
            getCellValue(row.getCell(6)), getCellValue(row.getCell(7)),
            getCellValue(row.getCell(8)), getCellValue(row.getCell(9)),
            getCellValue(row.getCell(10)), getCellValue(row.getCell(11))
        );
        count++;
    }
    return ResponseEntity.ok("✔ Successfully uploaded " + count + " users from
Excel!");
}
}

private ResponseEntity<?> processCSV(MultipartFile file) throws Exception {
    try (BufferedReader reader = new BufferedReader(new
InputStreamReader(file.getInputStream()))) {
        String line;
        int count = 0;
        reader.readLine(); // Skip header
        while ((line = reader.readLine()) != null) {
            // Regex handles commas inside quotes
            String[] data = line.split("(?=(?:[^\"]*"\"[^\"]*"")*\"[^\"]*$)", -1);
            if (data.length < 2) continue;

            saveUserFromData(data[0].trim().replace("\"", ""), data[1].trim().replace("\"",
""),
                (data.length > 2) ? data[2] : "STUDENT",
                (data.length > 3) ? data[3] : "",
                (data.length > 4) ? data[4] : "",

```

```

        (data.length > 5) ? data[5] : "",
        (data.length > 6) ? data[6] : "",
        (data.length > 7) ? data[7] : "",
        (data.length > 8) ? data[8] : "",
        (data.length > 9) ? data[9] : "",
        (data.length > 10) ? data[10] : "",
        (data.length > 11) ? data[11] : ""
    );
    count++;
}
return ResponseEntity.ok("✔ Successfully uploaded " + count + " users from
CSV!");
}
}

```

```

private void saveUserFromData(String name, String email, String roleStr, String
batchStr, String enroll,
    String company, String desig, String head, String skills,
    String git, String linked, String exp) {

```

```

    if (userRepository.existsByEmail(email)) return;

```

```

    User user = new User();
    user.setName(name);
    user.setEmail(email);

```

```

    // Safe Enum Conversion

```

```

    try {
        user.setRole(Role.valueOf(roleStr.toUpperCase().trim()));
    } catch (Exception e) {
        user.setRole(Role.STUDENT);
    }

```



```

// Clean Batch Year formatting (handles 2024.0 from Excel)
try {
    if (!batchStr.isEmpty()) {
        String cleanBatch = batchStr.contains(".") ? batchStr.split("\\.")[0] : batchStr;
        user.setBatchYear(Integer.parseInt(cleanBatch));
    }
} catch (Exception e) { /* Skip invalid years */ }

user.setEnrollmentNumber(enroll);
user.setCurrentCompany(company);
user.setDesignation(designation);
user.setHeadline(headline);
user.setSkills(skills);
user.setGithubUrl(githubUrl);
user.setLinkedinUrl(linkedinUrl);
user.setPastExperience(pastExperience);
user.setPasswordHash(passwordEncoder.encode("Bvicam@2025"));
user.setPasswordChanged(false);

userRepository.save(user);
}

private String getCellValue(Cell cell) {
    if (cell == null) return "";
    switch (cell.getCellType()) {
        case STRING: return cell.getStringCellValue().trim();
        case NUMERIC: return String.valueOf((long)cell.getNumericCellValue());
        case BOOLEAN: return String.valueOf(cell.getBooleanCellValue());
        default: return "";
    }
}
}
}

```

Feedback Controller:

```
package com.bvicam.campusconnect.controller;

import com.bvicam.campusconnect.entity.Feedback;
import com.bvicam.campusconnect.entity.User;
import com.bvicam.campusconnect.repository.FeedbackRepository;
import com.bvicam.campusconnect.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/feedback")
public class FeedbackController {

    @Autowired
    private FeedbackRepository feedbackRepository;

    @Autowired
    private UserRepository userRepository;

    // 1. Submit Feedback (Student/Alumni)
    @PostMapping
    public ResponseEntity<?> submitFeedback(@RequestBody Feedback feedback) {
        String email = SecurityContextHolder.getContext().getAuthentication().getName();
        User user = userRepository.findByEmail(email).orElseThrow();

        feedback.setSubmittedBy(user);
        feedbackRepository.save(feedback);
        return ResponseEntity.ok("Feedback submitted successfully!");
    }
}
```

```
// 2. View All Feedback (Admin Only)
@GetMapping
public List<Feedback> getAllFeedback() {
    return feedbackRepository.findAllOrderBySubmittedAtDesc();
}
}
```

User Controller:

```
package com.bvicam.campusconnect.controller;

import com.bvicam.campusconnect.dto.UserProfileDto;
import com.bvicam.campusconnect.entity.User;
import com.bvicam.campusconnect.repository.UserRepository;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.*;

import java.security.Principal;
import java.util.List;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserRepository userRepository;

    // Best Practice: Constructor Injection instead of @Autowired on field
    public UserController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

```

// 1. Get Current User Profile (Merged the two duplicates here)
@GetMapping("/me")
public ResponseEntity<?> getCurrentUser(Principal principal) {
    // specific check to avoid NullPointerException if security context is empty
    if (principal == null) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }

    return userRepository.findByEmail(principal.getName())
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

// Helper mapper
private UserProfileDto mapToDto(User user) {
    UserProfileDto dto = new UserProfileDto();
    dto.setId(user.getId());
    dto.setName(user.getName());
    dto.setEmail(user.getEmail());
    dto.setRole(user.getRole() != null ? user.getRole().name() : null);
    dto.setDepartmentName(user.getDepartment() != null ?
user.getDepartment().getName() : null);
    dto.setHeadline(user.getHeadline());
    dto.setCurrentCompany(user.getCurrentCompany());
    dto.setDesignation(user.getDesignation());
    dto.setSkills(user.getSkills());
    dto.setPastExperience(user.getPastExperience());
    dto.setLinkedInUrl(user.getLinkedInUrl());
    dto.setGithubUrl(user.getGithubUrl());
    dto.setNotifyMessages(user.getNotifyMessages());
    dto.setNotifyEvents(user.getNotifyEvents());
    dto.setNotifyJobs(user.getNotifyJobs());
    return dto;
}

```

```

// 2. Get All Users (For Directory)
@GetMapping
public List<UserProfileDto> getAllUsers() {
    return
userRepository.findAll().stream().map(this::mapToDto).collect(Collectors.toList());
}

// 3. Search Directory
@GetMapping("/search")
public List<UserProfileDto> searchDirectory(
    @RequestParam(required = false) String role,
    @RequestParam(required = false) Integer batch,
    @RequestParam(required = false) String q) {

    // Handle "ALL" as null to ignore the filter
    if ("ALL".equals(role)) role = null;
    // Handle empty search strings
    if (q != null && q.trim().isEmpty()) q = null;

    return userRepository.searchUsers(role, batch,
q).stream().map(this::mapToDto).collect(Collectors.toList());
}

// 4. Update Profile (LinkedIn Style)
@PutMapping("/profile")
public ResponseEntity<?> updateProfile(@RequestBody UserProfileDto profileDto) {
    // specific check to ensure authentication exists
    var authentication = SecurityContextHolder.getContext().getAuthentication();
    if (authentication == null || !authentication.isAuthenticated()) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }

    String email = authentication.getName();

```

```

// Use a safer way to retrieve user than .orElseThrow() without arguments
return userRepository.findByEmail(email).map(user -> {
    // Debugging
    System.out.println("Received Profile Update for: " + email);

    // Update fields from DTO only if they are not null
    if (profileDto.getHeadline() != null) user.setHeadline(profileDto.getHeadline());
    if (profileDto.getCurrentCompany() != null)
user.setCurrentCompany(profileDto.getCurrentCompany());
    if (profileDto.getDesignation() != null)
user.setDesignation(profileDto.getDesignation());
    if (profileDto.getSkills() != null) user.setSkills(profileDto.getSkills());
    if (profileDto.getPastExperience() != null)
user.setPastExperience(profileDto.getPastExperience());
    if (profileDto.getLinkedInUrl() != null)
user.setLinkedInUrl(profileDto.getLinkedInUrl());
    if (profileDto.getGithubUrl() != null)
user.setGithubUrl(profileDto.getGithubUrl());
    if (profileDto.getNotifyMessages() != null)
user.setNotifyMessages(profileDto.getNotifyMessages());
    if (profileDto.getNotifyEvents() != null)
user.setNotifyEvents(profileDto.getNotifyEvents());
    if (profileDto.getNotifyJobs() != null)
user.setNotifyJobs(profileDto.getNotifyJobs());

    userRepository.save(user);
    return ResponseEntity.ok("Profile Updated Successfully!");
}).orElse(ResponseEntity.status(HttpStatus.NOT_FOUND).body("User not
found"));
}
}

```

Web Sockets:

```
package com.bvicam.campusconnect.config;
```

```
import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import
org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import
org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer
;
```

```
@Configuration
```

```
@EnableWebSocketMessageBroker
```

```
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
```

```
    @Override
```

```
    public void registerStompEndpoints(StompEndpointRegistry registry) {
```

```
        // ✓ The main endpoint for connection
```

```
        // setAllowedOriginPatterns("*") is critical for deployment (Render/Heroku)
```

```
        registry.addEndpoint("/ws")
```

```
            .setAllowedOriginPatterns("*")
```

```
            .withSockJS();
```

```
    }
```

```
    @Override
```

```
    public void configureMessageBroker(MessageBrokerRegistry config) {
```

```
        // 1. Destinations the client can SUBSCRIBE to
```

```
        // /topic -> Public/Broadcast
```

```
        // /queue -> Private messages (standard convention)
```

```
        // /user -> User-specific events
```

```
        config.enableSimpleBroker("/topic", "/queue", "/user");
```

// 2. Destinations the client SENDS to (mapped to @MessageMapping in
Controllers)

```
config.setApplicationDestinationPrefixes("/app");
```

// 3. Prefix for User-specific messaging (e.g. /user/queue/messages)

```
config.setUserDestinationPrefix("/user");
```

```
}
```

```
}
```

Security Config:

```
package com.bvicam.campusconnect.config;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.security.authentication.AuthenticationManager;
```

```
import
```

```
org.springframework.security.config.annotation.authentication.configuration.Authenticati  
onConfiguration;
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
```

```
import
```

```
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
```

```
import org.springframework.security.config.http.SessionCreationPolicy;
```

```
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
```

```
import org.springframework.security.crypto.password.PasswordEncoder;
```

```
import org.springframework.security.web.SecurityFilterChain;
```

```
import
```

```
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter
```

```
;
```

```
import org.springframework.web.cors.CorsConfiguration;
```

```
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
```

```
import java.util.List;
```



```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .headers(headers -> headers.frameOptions(frameOptions ->
frameOptions.disable()))
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                // ✓ ADDED "/favicon.ico" here to fix the 403 error
                .requestMatchers(
                    "/",
                    "/index.html",
                    "/login.html",
                    "/dashboard.html",
                    "/app.js",
                    "/dashboard.js",
                    "/components/**",
                    "/css/**",
                    "/js/**",
                    "/ws/**",
                    "/favicon.ico",
                    "/ws/**"
                )
                .permitAll()
                .requestMatchers("/api/auth/**").permitAll()
            )
    }
}

```

```

        .requestMatchers("/error").permitAll()
        .anyRequest().authenticated()
    )
    .addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

@Bean
public UrlBasedCorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();

    // Allow specific patterns + Credentials
    config.setAllowedOriginPatterns(List.of("*"));
    config.setAllowCredentials(true);

    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
    config.setAllowedHeaders(List.of("*"));

    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean

```

```

    public AuthenticationManager authenticationManager(AuthenticationConfiguration
config) throws Exception {
        return config.getAuthenticationManager();
    }
}

```

JWT authentication:

```

package com.bvicam.campusconnect.config;

import com.bvicam.campusconnect.service.CustomUserDetailsService;
import com.bvicam.campusconnect.util.JwtUtil;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import
org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

```

```

@Autowired
private CustomUserDetailsService userDetailsService;

@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain filterChain)
    throws ServletException, IOException {

    // 1. Get the Authorization Header
    String authHeader = request.getHeader("Authorization");
    String token = null;
    String username = null;

    // 2. Check if it starts with "Bearer "
    if (authHeader != null && authHeader.startsWith("Bearer ")) {
        token = authHeader.substring(7); // Remove "Bearer " prefix
        try {
            // 3. Extract Username from Token
            username = io.jsonwebtoken.Jwts.parserBuilder()
                .setSigningKey(jwtUtil.getSignKey()) // Uses the secret key from JwtUtil
                .build()
                .parseClaimsJws(token)
                .getBody()
                .getSubject();
        } catch (Exception e) {
            System.out.println("Invalid Token: " + e.getMessage());
        }
    }

    // 4. Validate Token and set Authentication context
    if (username != null && SecurityContextHolder.getContext().getAuthentication()
== null) {
        UserDetails userDetails = userDetailsService.loadUserByUsername(username);
    }
}

```

```

        // In a real app, validate expiration here too. For now, we trust the parsing.
        UsernamePasswordAuthenticationToken authToken = new
UsernamePasswordAuthenticationToken(
            userDetails, null, userDetails.getAuthorities());

        authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

        // This is the "Stamp of Approval" - User is now logged in for this request
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }

    filterChain.doFilter(request, response);
}
}

```

Data Seeder:

```

package com.bvicam.campusconnect.config;

import com.bvicam.campusconnect.entity.Department;
import com.bvicam.campusconnect.entity.Job;
import com.bvicam.campusconnect.entity.Post;
import com.bvicam.campusconnect.entity.Role;
import com.bvicam.campusconnect.entity.User;
import com.bvicam.campusconnect.repository.DepartmentRepository;
import com.bvicam.campusconnect.repository.JobRepository;
import com.bvicam.campusconnect.repository.PostRepository;
import com.bvicam.campusconnect.repository.UserRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

```

```

import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.List;

@Configuration
public class DataSeeder {

    @Bean
    CommandLineRunner initDatabase(UserRepository userRepo,
                                   JobRepository jobRepo,
                                   PostRepository postRepo,
                                   PasswordEncoder passwordEncoder,
                                   DepartmentRepository deptRepo) {
        return args -> {
            // 1. Create ADMIN
            if (!userRepo.existsByEmail("admin@bvicam.in")) {
                User admin = new User();
                admin.setName("Super Admin");
                admin.setEmail("admin@bvicam.in");
                admin.setPasswordHash(passwordEncoder.encode("admin123")); // Password:
admin123
                admin.setRole(Role.ADMIN);
                userRepo.save(admin);
                System.out.println("✔ Created Admin: admin@bvicam.in / admin123");
            }

            // 2. Create ALUMNI
            if (!userRepo.existsByEmail("alumni@bvicam.in")) {
                User alumni = new User();
                alumni.setName("Amit Verma");
                alumni.setEmail("alumni@bvicam.in");
                alumni.setPasswordHash(passwordEncoder.encode("1234")); // Password:
1234
            }
        };
    }
}

```

```

        alumni.setRole(Role.ALUMNI);
        alumni.setBatchYear(2020);
        alumni.setSkills("Java, Spring Boot, AWS");
        userRepo.save(alumni);
        System.out.println("✔ Created Alumni: alumni@bvicam.in / 1234");

        // Add a Dummy Job for this Alumni
        Job job = new Job();
        job.setTitle("Junior Java Developer");
        job.setCompany("TCS");
        job.setLocation("Noida");
        job.setDescription("Looking for freshers with good Java knowledge.");
        job.setApplyLink("careers@tcs.com");
        job.setPostedBy(alumni);
        jobRepo.save(job);
    }

    // 3. Create STUDENT
    if (!userRepo.existsByEmail("student@bvicam.in")) {
        User student = new User();
        student.setName("Aadish Goel");
        student.setEmail("student@bvicam.in");
        student.setPasswordHash(passwordEncoder.encode("1234")); // Password:
1234

        student.setRole(Role.STUDENT);
        student.setEnrollmentNumber("0012024");
        userRepo.save(student);
        System.out.println("✔ Created Student: student@bvicam.in / 1234");

        // Add a Dummy Wall Post
        Post post = new Post();
        post.setContent("Just finished my minor project! Excited for the final exams.");
        post.setAuthor(student);

```

```

        post.setLikes(5);
        postRepo.save(post);
    }
    if (deptRepo.findByCode("MCA").isEmpty()) {
        deptRepo.save(new Department("Master of Computer Applications", "MCA"));
        System.out.println("✔ Created Department: MCA");
    }

    // Check if BJMC exists, if not, create it
    if (deptRepo.findByCode("BJMC").isEmpty()) {
        deptRepo.save(new Department("Bachelors of Journalism", "BJMC"));
        System.out.println("✔ Created Department: BJMC");
    }
};
}
}

```

Event DTO:

```

package com.bvicam.campusconnect.dto;

import com.fasterxml.jackson.annotation.JsonFormat;
import java.time.LocalDateTime;

public class EventDTO {
    private Long id;
    private String title;
    private String description;
    private String location;

    // ✔ Ensures date is sent as a String, not an array

```



```

@JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "yyyy-MM-
dd'T'HH:mm")
private LocalDateTime dateTime;

// ✓ CHANGED: Wrapper classes (Boolean, Integer) handle nulls safely
private Boolean attending;
private Integer participantCount;

// 1. Default Constructor
public EventDTO() {
}

// 2. All-Arguments Constructor
public EventDTO(Long id, String title, String description, String location,
LocalDateTime dateTime, Boolean attending, Integer participantCount) {
    this.id = id;
    this.title = title;
    this.description = description;
    this.location = location;
    this.dateTime = dateTime;
    this.attending = attending;
    this.participantCount = participantCount;
}

// 3. Getters and Setters

public Integer getParticipantCount() {
    return participantCount;
}

public void setParticipantCount(Integer participantCount) {
    this.participantCount = participantCount;
}

```

```
public Long getId() {  
    return id;  
}
```

```
public void setId(Long id) {  
    this.id = id;  
}
```

```
public String getTitle() {  
    return title;  
}
```

```
public void setTitle(String title) {  
    this.title = title;  
}
```

```
public String getDescription() {  
    return description;  
}
```

```
public void setDescription(String description) {  
    this.description = description;  
}
```

```
public String getLocation() {  
    return location;  
}
```

```
public void setLocation(String location) {  
    this.location = location;  
}
```

```
public LocalDateTime getDateTime() {
```

```

        return dateTime;
    }

    public void setDateTime(LocalDateTime dateTime) {
        this.dateTime = dateTime;
    }

    // Changed getter name from isAttending() to getAttending() for Boolean wrapper
    convention
    public Boolean getAttending() {
        return attending;
    }

    public void setAttending(Boolean attending) {
        this.attending = attending;
    }
}

```

Typing message:

```
package com.bvicam.campusconnect.dto;
```

```

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

```

```
@Data
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```

public class TypingMessage {
    private String senderEmail;
    private String receiverEmail;
    private boolean isTyping;
}

```

```
}
```

JWT Util:

```
package com.bvicam.campusconnect.util;
```

```
import io.jsonwebtoken.Jwts;
```

```
import io.jsonwebtoken.SignatureAlgorithm;
```

```
import io.jsonwebtoken.io.Decoders;
```

```
import io.jsonwebtoken.security.Keys;
```

```
import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.stereotype.Component;
```

```
import java.security.Key;
```

```
import java.util.Date;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
@Component
```

```
public class JwtUtil {
```

```
@Value("${app.jwt.secret:5367566B59703373367639792B423F4528482B4D62516554  
68576D5A71347437}")
```

```
    private String secret;
```

```
    public String generateToken(String userName) {
```

```
        Map<String, Object> claims = new HashMap<>();
```

```
        return createToken(claims, userName);
```

```
    }
```

```
    private String createToken(Map<String, Object> claims, String subject) {
```

```

        return Jwts.builder()
            .setClaims(claims)
            .setSubject(subject)
            .setIssuedAt(new Date(System.currentTimeMillis()))
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10)) //
10 Hours validity
            .signWith(getSignKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public Key getSignKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}

```

Campus Connect Application. Java:

```

package com.bvicam.campusconnect;

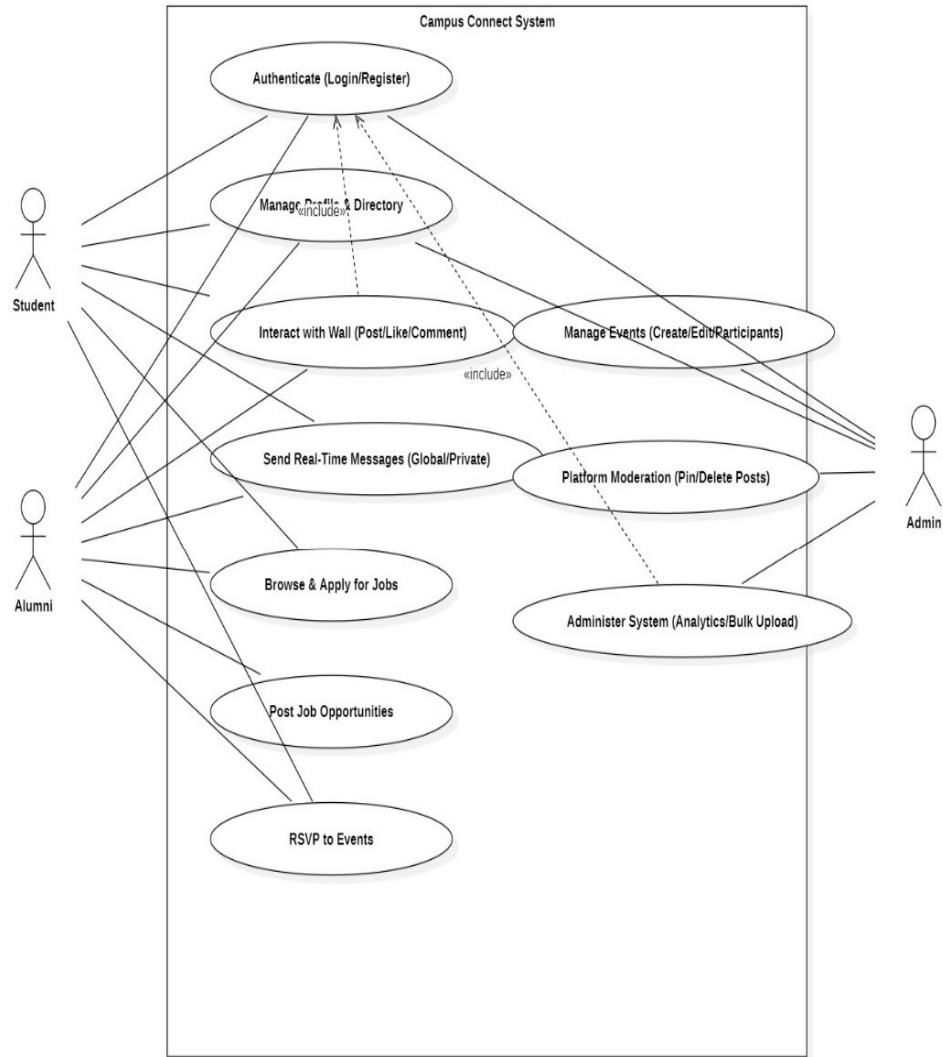
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableAsync;

@SpringBootApplication
@EnableAsync
public class CampusConnectApplication {
    public static void main(String[] args) {
        SpringApplication.run(CampusConnectApplication.class, args);
    }
}

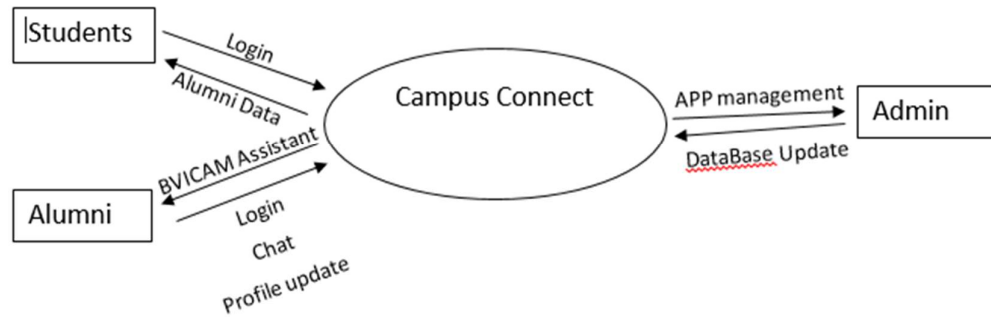
```

CHAPTER 3: FUNCTIONALITIES

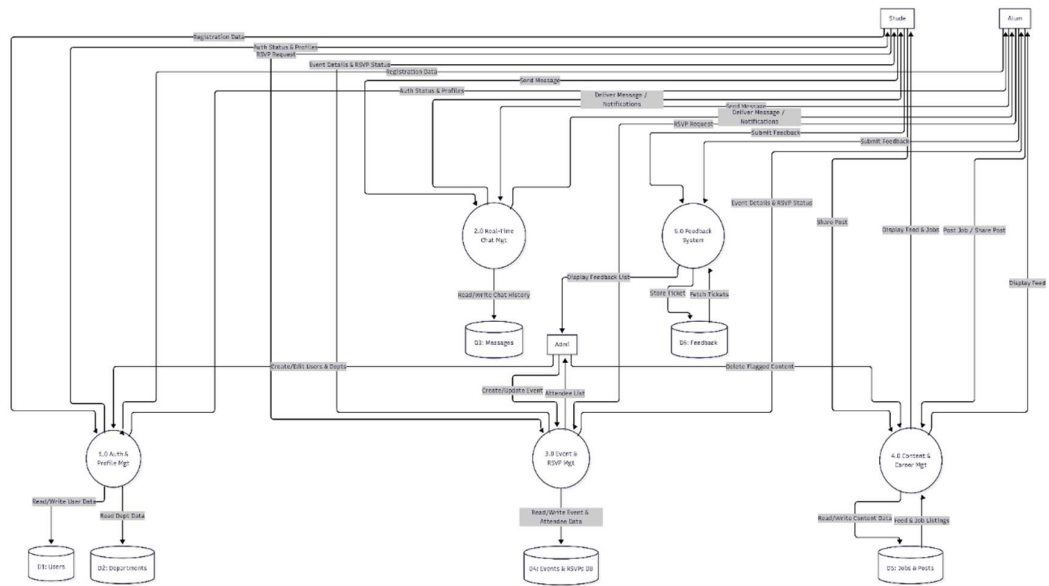
Use Case Diagram



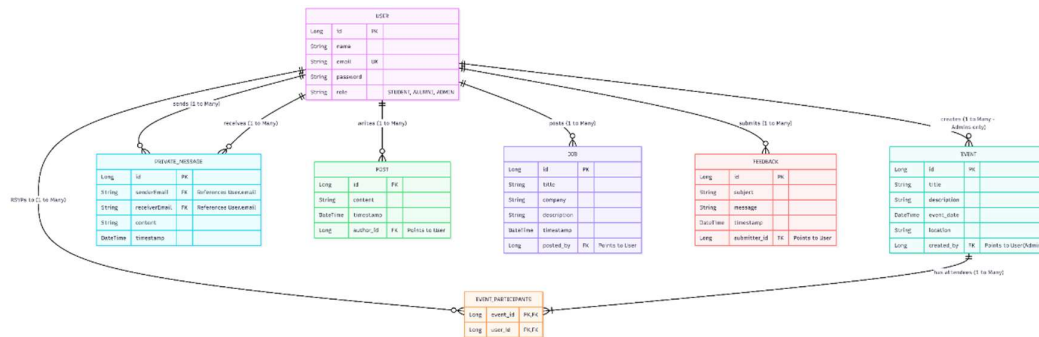
Data Flow Diagram Level 0



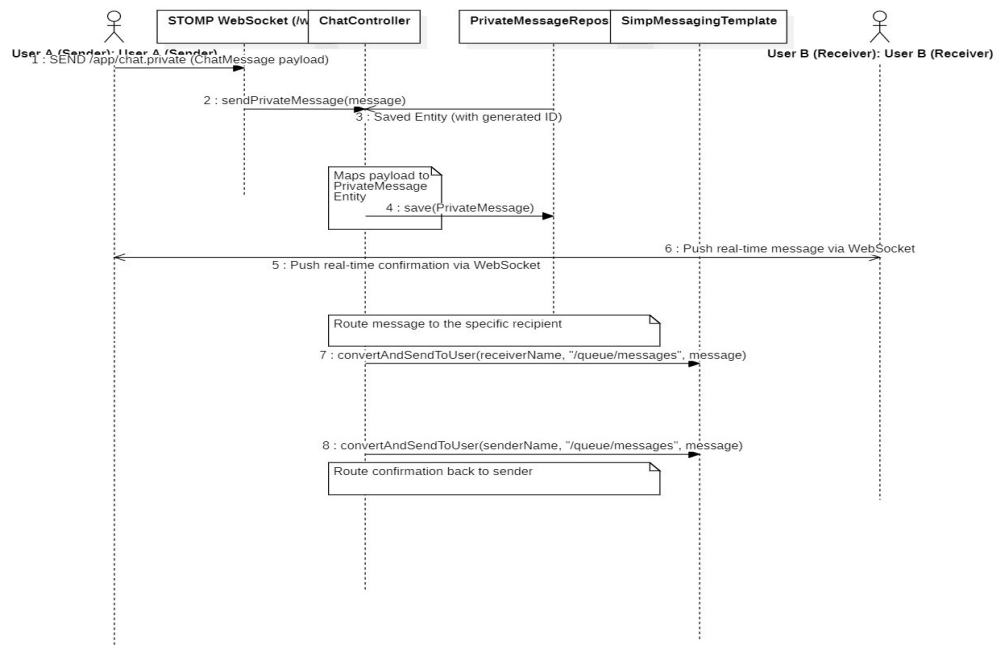
DFD Level 1



Entity-Relationship Diagram



Sequence Diagram



Screenshots

Login:

Campus Connect

Sign in to connect with students, alumni, jobs, events, and your campus community.

EMAIL ADDRESS

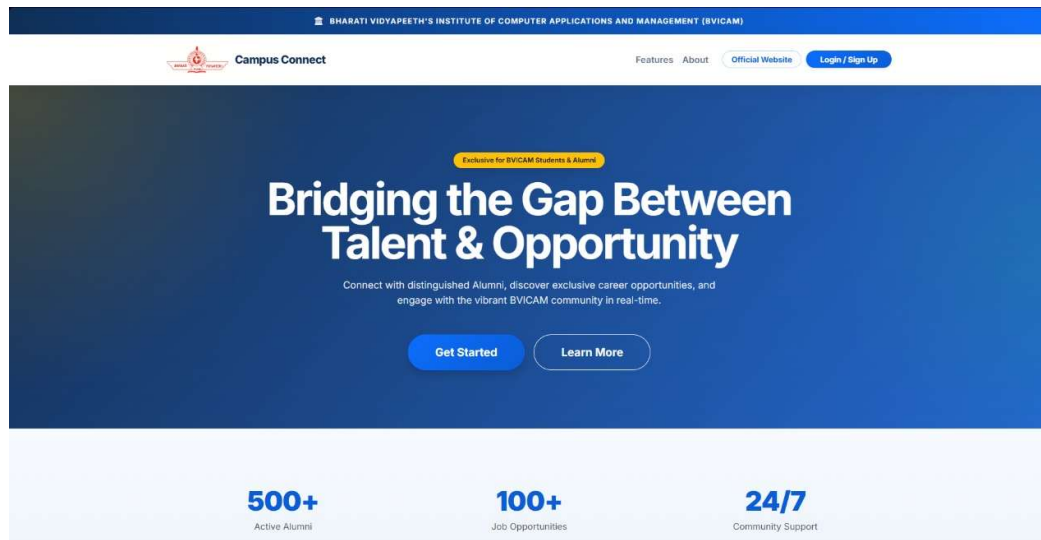
name@bvicam.in

PASSWORD

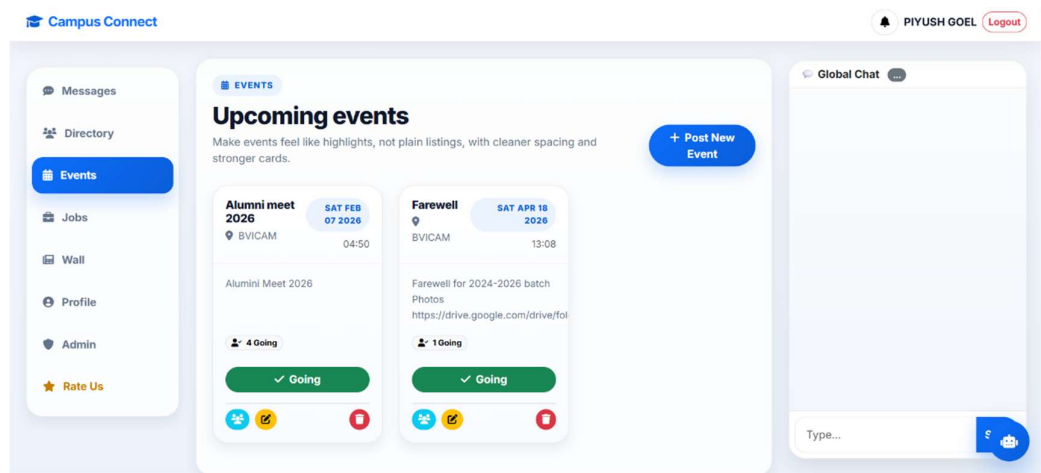
Sign In

[← Back to Home](#)

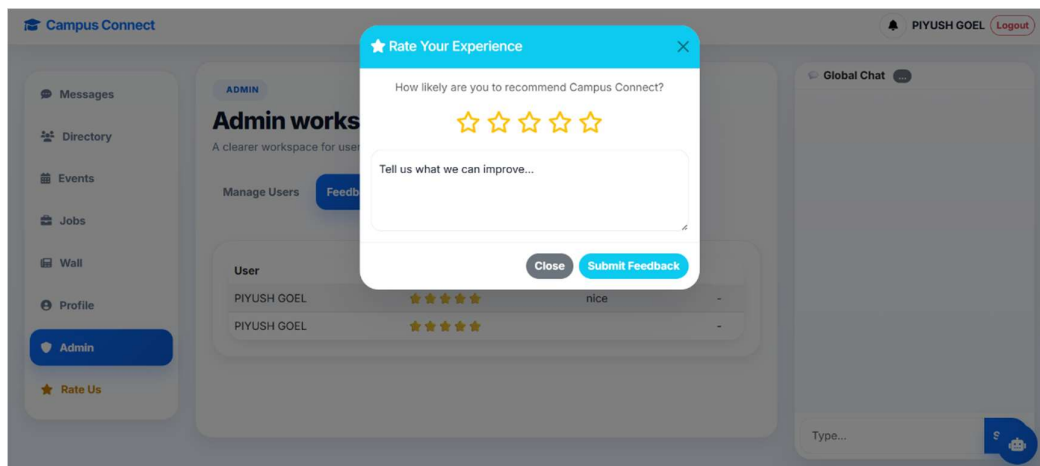
Dashboard:



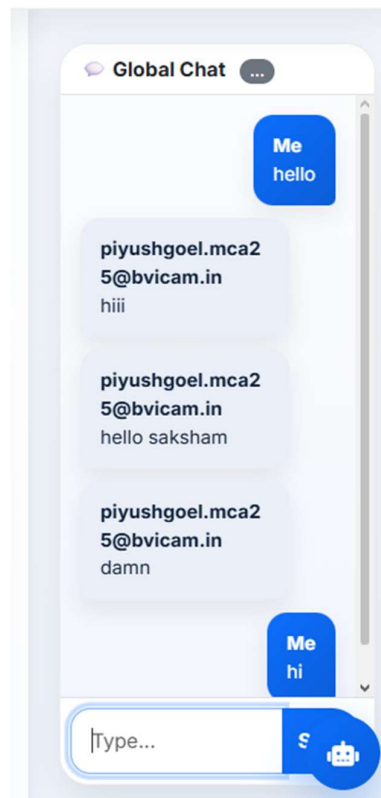
Events:



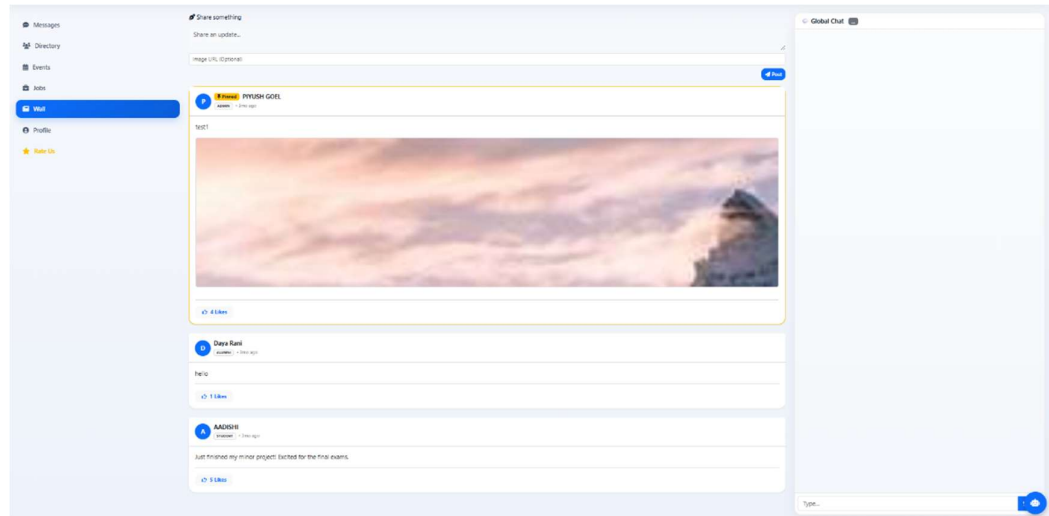
Feedback:



Global Chat:



Memory Wall:



Profile:



 SAKSHAM [Logout](#)

 Messages

 Directory

 Events

 Jobs

 Wall

 **Profile**

 Rate Us

Edit Profile

Save Changes

Headline
Aspiring Software Engineer

Current Company
TCS

Designation
Web Developer

Skills
C++, DSA

Experience
Summer Intern

LinkedIn
linkedin.com/in/saksham817

GitHub
github.com/saksham59

Notification Preferences

☐ Messages

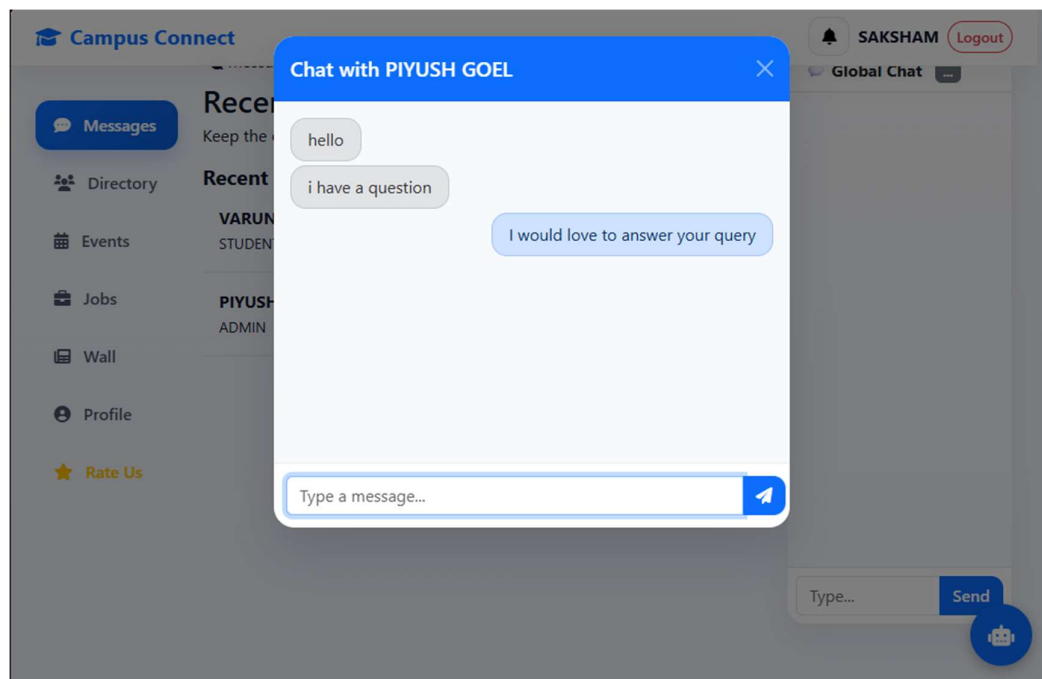
☐ Events

☐ Jobs

Global Chat

Type... 

Directory:



CHAPTER 4: CONCLUSION AND FUTURE SCOPE

- Campus connect, a Java based application that could bridge the gap between the current students and alumni through a user friendly digital platform. This application can help shaping the future of new students by providing them guidance from their senior alumni working in various companies in various roles. Thus the Experienced alumni can help them choose their career path and help them with career maps and from their own experience guide them on what to do and what to avoid.
- Through this application the seniors can enquire about various kinds of information about the institutes current state such as events and such through the use of Static chat-bot.
- The alumni can use this application to share their stories and pictures of their time in the institute through “Alumni Memory wall” show casing their fun stories and educational experiences through pictures and text.
- The current students can enquire about exciting job opportunities, upcoming events, alumni details and so on through the use of Static chat-bot.
- The students can access the Alumni profile and gather information such as contact information and the courses their alumni took when they were studying in the institute.
- There is a community chat in our application which can be used by all the users of our app, here everyone can talk as a community and interact with each other, get to know their community, etc.
- This project was made using MySQL and Java Programming, we used the Apache Netbeans software for developing an interactive GUI and MySQL for a well maintained and easy to edit DataBase.
- The users of our project can even give their Feedback regarding their experience of using our app and improvements can be made accordingly.

- The developed application was made in different parts using the prototype and Iterative model of software development, we made primarily 5 models for this project which contains:

- [Events](#)
- [Dashboard](#)
- [Global Messages](#)
- [Direct Messages](#)
- [Memory Wall.](#)
- [Feedback](#)
- [Jobs](#)

Future scope:

The project can be enhanced by incorporating various complex modules such as:

- Real time video or audio chat between the senior Alumni and students by setting up meetings between them.
- Digital portfolio system that will hold alumni resume and work experience and activities they are taking a part in to motivate juniors to get out of their shoes and look outside the box.
- Increasing the scope of the data base by Cloud deployment as the data increases and complexity increases we can outsource the database on a cloud such that it can be accessed by anyone with proper credentials.
- Study the career path of alumni and use it to guide new students through AI powered career guidance system and give them a proper action plan and make them industry ready.
- We can create a meeting environment in which one can schedule seminars and such to guide students.

- **BIBLIOGRAPHY**

Books:

- Herbert Schildt, “Java: The complete reference”.

Websites:

- <https://bvicam.in/UploadDocuments/CourseMaterial/MCA%20109%2UNIT%20I.pdf>
- <https://learn.oracle.com/ols/learning-path/oracle-java-foundations/88323/79726>
- <https://editor.plantuml.com/uml/SoWkIImgAStDuNBCoKnELT2rKt3AJx9IS2mjoKZDAybCJYp9pCzJ24ejB4qjBk42oYde0jM05MDHLLoGdrUSoeLkM5u-K5sHGY9sGo6ARNHr2QY66kwGcfS2SZ00>

- **APPENDICES**

A-1 Apache Maven:

Apache Maven is a build tool generally used for java applications like in this case, it allows one to manage the project and build automation tools. Gives a standard structure to the whole project and manages various dependencies.

A-2 Spring Boot:

Spring boot is another automation tool generally used for creating database connections automatically, it consists of an embedded server and lets one to develop stand alone production grade applications. It lets one to speed up the development process of the project and is great for creating micro services.